Session 4 — Python environments: venv, pip, uv, conda

Git, Python Environments & Coding Best Practices

Guillaume Leveque

Session 4 of 4

Master of Data & AI — MSA-DATB07-03

The problem (5 min)

Bob sends you his project

```
env-demo/
|-- report.py
|-- sales.csv
`-- requirements.txt
```

`report.py` reads `sales.csv` and prints the revenue per region. On Bob's laptop, it works.

```python
import pandas as pd
from tabulate import tabulate

sales = pd.read_csv("sales.csv")
sales["revenue"] = sales["quantity"] * sales["unit_price"]
by_region = sales.groupby("region")["revenue"].sum().round(2)
print(tabulate(by_region.items(), headers=["region", "revenue"]))
```

On your laptop:

```
$ python report.py
ModuleNotFoundError: No module named 'pandas'
```

`pandas` (read tables) and `tabulate` (print tables) are **libraries**: code written by others, that Python does not have by itself. They are installed with `pip`.

Why not just “pip install pandas”?

without environments

One Python for everything

pandas 2? pandas 3?
150 libraries piled up
since you bought the laptop

with one environment per project

project A
pandas 2

project B
pandas 3

- Every project of your laptop shares **one** list of libraries. Project A needs pandas 2, project B pandas 3: one of them breaks.
- Which versions did Bob use? Nobody wrote it down: “it works on my machine”.

The answer has two parts: **a virtual environment** (a private list of libraries per project), and **requirements.txt** (the written list).

A virtual environment (15 min)

A virtual environment: a private Python for one project

A folder `.venv/` inside the project, with:

- its own `python`
- its own list of libraries, **empty at the start**
- a script `activate`: "in this terminal, use this `python`"

What you install in it stays in this project.

Rules

- one project, one `.venv`, inside the project
- never committed: `.venv/` in `.gitignore`
- broken? delete it, create it again: one minute

Step 1 — Create it, activate it

```
cd env-demo                          # Bob's project
python -m venv .venv                 # create the environment (macOS: python3 -m venv .venv)
source .venv/Scripts/activate        # activate it: Windows, Git Bash
source .venv/bin/activate            # activate it: macOS (run only the line of your system)
which python                         # which python is used now?
```

```
(.venv)
$ which python
/c/Users/alice/courses/git/demo/env-demo/.venv/Scripts/python
```

- `python -m venv .venv`: run Python's module `venv`, create the folder `.venv`.
- `(.venv)` in the prompt: active *in this terminal*. A new terminal starts without it: activate again. `deactivate` leaves it.

Step 2 — The new environment is empty

```
python -m pip list              # what is installed in this environment?
python report.py
```

```
Package Version
------- -------
pip     24.2
```

```
Traceback (most recent call last):
  File "report.py", line 3, in <module>
    import pandas as pd
ModuleNotFoundError: No module named 'pandas'
```

Normal: a new environment knows **no** library, only `pip`. That is the point: nothing from the rest of the laptop leaks in. Now we install exactly what the project needs.

VS Code and the environment

- `Ctrl+Shift+P` → *Python: Select Interpreter* → the one in `.venv`. VS Code then uses it for the Run button and activates it in every new terminal.
- The bottom-right corner of VS Code shows the interpreter in use: look there first when you see "No module named . . . ".

Pitfall — Scripts or bin

The `activate` script is in `.venv/Scripts/` on Windows, in `.venv/bin/` on macOS. Same command otherwise; `Tab` completes the right one.

Pitfall — installed, but "No module named ..."

You installed with one Python and run with another. Always `python -m pip`: it is the pip of the python you run.

requirements.txt (15 min)

What is requirements.txt?

```
$ cat requirements.txt
pandas==3.1.0
tabulate==0.10.0
```

- A plain text file, at the root of the project.
- One library per line: `name==version`.
- The **shopping list** of the project: what to install so that the code runs.
- Committed with the code: the whole team has the same list.

`pandas==3.1.0`	exactly this version: the same on every machine, today and next year
`pandas>=3.1`	this version or a newer one: two installs a month apart may differ

Step 3 — Install what the list says

```
python -m pip install -r requirements.txt     # -r = read the list from this file
```

```
Installing collected packages: tzdata, tabulate, six, numpy, python-dateutil, pandas
Successfully installed numpy-2.5.4 pandas-3.1.0 python-dateutil-3.0.0 six-1.17.0 tabulate-0.10.0 tzdata
    -2026.5
```

- The two libraries of the list, with *exactly* the versions written in it.
- And the libraries *they* need (numpy for pandas, ...): pip finds them by itself.
- Everything goes into `.venv`, nowhere else.

Step 4 — It works

```
python report.py
```

```
region      revenue
--------  ---------
East          240
North         284
South         530.5
West          437
```

Same code, same libraries, same versions as Bob: same result. You did not need to ask Bob anything: the project carried its own list.

Step 5 — How Bob wrote the list: pip freeze

```
python -m pip freeze              # every library of the environment, name==version
```

```
numpy==2.5.4
pandas==3.1.0
python-dateutil==3.0.0
six==1.17.0
tabulate==0.10.0
tzdata==2026.5
```

- Bob kept the lines of the libraries he *imports*: `pandas` and `tabulate`. pip finds the others by itself.
- Or the whole list in one command: `python -m pip freeze > requirements.txt` (`>`: write into the file, session 1). Stricter: every version is fixed.

The recipe: a new teammate, on day one

1. `git clone <address>` and `cd` into the project
2. `python -m venv .venv` — create the environment
3. `source .venv/Scripts/activate` — (macOS: `.venv/bin/activate`)
4. `python -m pip install -r requirements.txt` — install the list
5. run the code, run the tests — it works

Committed	Never committed
the code, `requirements.txt`, `.gitignore`	`.venv/`: listed in `.gitignore`, recreated on each machine

If step 5 fails with "No module named ...", a library is missing from `requirements.txt`.

requirements.txt in a team

Alice writes `plot.py`, which starts with `import matplotlib`. She must:

1. install it in her environment: `python -m pip install matplotlib`
2. add its line at the end of `requirements.txt`: `matplotlib==3.11.2`
3. commit `plot.py` **and** `requirements.txt` together, then push.

If she forgets step 2, Bob pulls `plot.py`, runs it, and gets `ModuleNotFoundError: No module named 'matplotlib'`.

Bob, after `git pull`: `python -m pip install -r requirements.txt`. `git pull` brings files, not libraries.

uv and conda (15 min)

Three tools, one idea

	pip + venv	**uv**	**conda**
what	installs Python libraries	pip + venv in one tool, much faster; can install Python itself	installs libraries, Python itself, and non-Python libraries
environment	`python -m venv .venv`	`uv venv`	`conda create -n name`
the list	`requirements.txt`	`requirements.txt`, or `pyproject.toml` + `uv.lock`	`environment.yml`
comes with	Python	`python -m pip install uv`	Anaconda or Miniconda
you meet it	everywhere	new projects	data science, GPUs, scientific libraries

Same idea every time: **an isolated environment per project**, and **a file that describes it**, committed with the code.

Step 6 — The same, with uv

```
deactivate                          # leave the environment
python -m pip install uv            # install uv itself, once per computer
rm -r .venv                         # start again from nothing
uv venv                             # = python -m venv .venv
source .venv/Scripts/activate       # macOS: source .venv/bin/activate
uv pip install -r requirements.txt  # = python -m pip install -r requirements.txt
python report.py                    # the same table
```

```
Creating virtual environment at: .venv
Resolved 6 packages in 8ms
Installed 6 packages in 974ms
```

Under one second: uv downloads in parallel and keeps a cache. Same `requirements.txt`, same `.venv`: in a team, each member may use pip or uv.

Choose the Python version

With `python -m venv`, the environment takes the Python that created it: to get Python 3.12, you first need Python 3.12 installed on the laptop. uv can download the Python version you ask for.

```
uv python list                  # the versions on your laptop, and those uv can download
uv venv --python 3.12           # an environment with Python 3.12 (downloaded if missing)
uv python pin 3.12              # write 3.12 into .python-version: this project uses 3.12
```

```
$ uv venv --python 3.12
Using CPython 3.12.7 interpreter at: C:\Users\alice\...\Python312\python.exe
Creating virtual environment at: .venv
```

- `.python-version` is a one-line file, committed: every teammate's `uv venv` uses the same version.
- Which version? The one the project asks for (README, `.python-version`); otherwise a recent one, 3.12 or newer.

uv for a whole project

```
uv init weather          # a new project: pyproject.toml, .python-version, a Git repository
cd weather
uv add pandas            # install pandas AND write it in pyproject.toml and uv.lock
uv run python -c "import pandas; print(pandas.__version__)"   # run in the environment, no
    activation
uv sync                  # a teammate, after git clone: install exactly what uv.lock says
```

`pyproject.toml`	what the project needs, loosely (`pandas>=3.1.0`); written by `uv add`
`uv.lock`	the **exact** version of every library, dependencies included; written by `uv add`

The same idea as `requirements.txt` with `==`, but uv keeps the list up to date by itself.

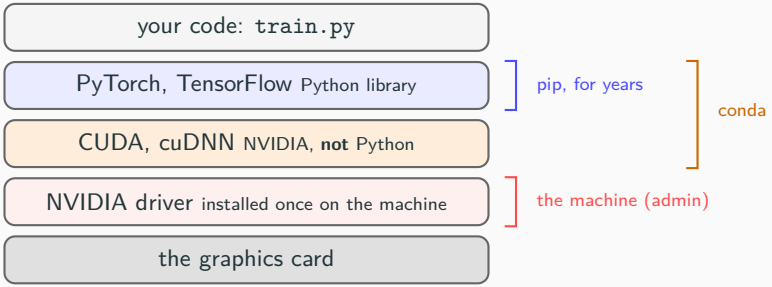
conda: why it exists

pip and uv install **Python libraries** (from PyPI). conda installs **any software** into the environment: Python libraries, but also

- **Python itself**: `conda create -n old python=3.9` (uv can do it too: previous slides);
- **programs and libraries that are not Python**: maps (GDAL), fast maths (MKL), R, compilers... and NVIDIA's **CUDA**, for the GPU.

A conda environment is a whole private toolbox, not only a list of Python libraries. That is exactly what the GPU needs.

The GPU: four layers that must fit together



Each layer needs the right version of the one below: PyTorch 2.x is built for a given CUDA, and that CUDA needs a recent enough driver. One wrong version, and `torch.cuda.is_available()` answers `False`: the code runs on the processor, much slower.

conda in practice

```
conda create -n sales python=3.12 pandas   # an environment named "sales", with its own
    Python
conda activate sales                       # use it in this terminal
conda install tabulate                     # install into it
conda env export > environment.yml         # the list, like pip freeze
conda env create -f environment.yml        # recreate it on another machine
```

- The environments live in one central folder, not inside the project: you call them by their name.
- `environment.yml` plays the role of `requirements.txt`: committed with the code.
- In a conda environment, install with `conda` first; `pip` only for what conda does not have.

Which one should I use?

- **The project already has a file**: use its tool. `requirements.txt` → pip or uv; `uv.lock` → uv; `environment.yml` → conda.
- **A new project of yours**: uv (or pip + venv, the same thing, slower).
- **GPU**: follow the install page of PyTorch or TensorFlow (pip, uv or conda). conda when the team or the server already uses it, or without admin rights.
- **A library that is not Python** (GDAL, R…): conda.
- **Always**: one environment per project, its list committed, tested on a fresh clone.

A tidy project (5 min)

The layout of a project

```
msa-datb07-team/
|-- README.md
|-- requirements.txt
|-- .gitignore      (.venv/ inside)
|-- data/           (input)
|-- notebooks/      (exploration)
|-- outputs/        (generated)
|-- src/sales_analysis/
|   |-- __init__.py
|   |-- loader.py
|   `-- metrics.py
`-- tests/
```

- `README.md`: what the project does, how to install it (the recipe), how to run it.
- `data/`: the raw data is read, never edited.
- `src/`: the code that is reused; `tests/`: small programs that check it (next slide).
- `outputs/`: generated (charts, tables), ignored by Git: it can be produced again.

tests/: small programs that check your code

A test does not fix the revenue of the company: it checks the *formula* (quantity × price) on two made-up rows whose answer you know by hand.

```python
# tests/test_metrics.py
def test_total_revenue():
    rows = [{"quantity": 2, "unit_price": 10.0},     # 2 x 10   = 20
            {"quantity": 1, "unit_price": 25.5}]     # 1 x 25.5 = 25.5
    assert total_revenue(rows) == 45.5               # 20 + 25.5, by hand
```

Next month, a teammate edits `total_revenue` and forgets the quantity. `python -m pytest` catches it at once:

```
E       AssertionError: assert 35.5 == 45.5
FAILED tests/test_metrics.py::test_total_revenue
1 failed in 0.07s
```

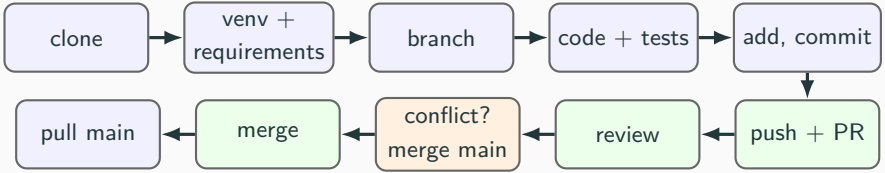
Without the test, the wrong totals go into the report and nobody sees it. With it, the mistake is found before the pull request.

Notebooks in a repository

- A notebook stores its outputs (tables, images) inside the file: every run changes it, the diffs are unreadable and the conflicts painful.
- **Clear the outputs before committing** (VS Code: *Clear All Outputs*).
- Notebooks *explore*. Code that is reused moves to `src/`, and the notebook imports it.
- The notebook uses the environment too: in VS Code, pick the kernel of `.venv`.

Wrap-up (5 min)

Twelve hours in one picture



then the next task: a new branch

Session 1: the terminal and the commit. Session 2: branches, merges, conflicts. Session 3: push, pull requests, reviews. Session 4: environments. Blue: on your laptop; green: on GitHub; orange: when two of you changed the same lines.

Safe or not: the final summary

command	effect	risk
`status`, `diff`, `log`, `branch`, `remote -v`	read only	**safe**
`add`, `commit`, `switch`, `merge`, `revert`, `push`, `pull`	the history grows	**safe** (merge and pull may conflict)
`merge --abort`	back to the state before the merge	**safe**
`restore file`	throw away the edits not committed	**destroys work** (those edits)
`reset --hard`	drop commits and edits	**destroys work** (only for commits never pushed)
`push --force`	replace the branch on GitHub	**never on shared history**
`rm -r .venv`	delete the environment	**safe** (recreate it from `requirements.txt`)

The quiz

- One multiple-choice quiz on the four sessions.
- The kinds of question: what does this command do; which command gives this result; read a folder tree, a `git status`, a commit graph (`git log --oneline --graph --all`), an error message.

The appendices of the slides are not in the quiz.

Before the quiz

Practise: everything is on the course's GitHub

`https://github.com/msa-datb07-git/course`

- `shell-practice`: short exercises on the terminal and Git, with their solutions;
- Learn Git Branching: Git in your browser, nothing to install;
- the TD repositories, to replay the TDs.

Read again

The slides of the four sessions (the "Check yourself" and "Read this repository" slides), the TDs and their answer keys, today's Kahoot.

Appendix: for the curious

Appendix: tools that check your code

Not in the quiz

These tools exist in most professional projects. You will meet them in your internships.

black	a *formatter*: rewrites the layout of the code (spaces, quotes, line breaks), the same way for everybody
ruff	a *linter*: flags probable mistakes (an import never used, a name that does not exist)
mypy	a *type checker*: `total(rows: list) -> float` called with a string? Error before running
pre-commit	runs these tools at every `git commit`, and refuses the commit if one fails
GitHub Actions	runs them, and the tests, on GitHub for every pull request: a red cross blocks the merge

Appendix — What a tool catches, and what only a reviewer can

```python
# before black
def f(a,b = 1):
    return {'x':a,"y" :b}
```

```python
# after black
def f(a, b=1):
    return {"x": a, "y": b}
```

Tools

- layout, quotes, line length
- unused or undefined names
- wrong types
- “do the tests still pass?”

A reviewer

- is the name honest? (`monthly_revenue` that computes a yearly one)
- is the result right? the edge cases handled?
- do the tests check something useful?

The tools take the boring part, identically for everyone; the reviewer’s attention goes to the right column.